In the Name of God

Digital Image Processing

Prof. Zohreh Azimifar

Homework 1

Topics: Point Operations, Histogram Processing, and Pixel Resolution

Submission Format: PDF Report + Source Code (.zip)

Section 0: Warm-up – Array Indexing (Optional - Bonus points)

Image processing relies heavily on matrix manipulations. This section serves as a mathematical warm-up.

Task:

Write a program that receives two 2D matrices as input: a smaller matrix S of size $N \times M$, and a larger matrix L of size $H \times W$.

Your task is to embed the smaller matrix S into the larger matrix L in a rotated, diamond-like checkerboard pattern.

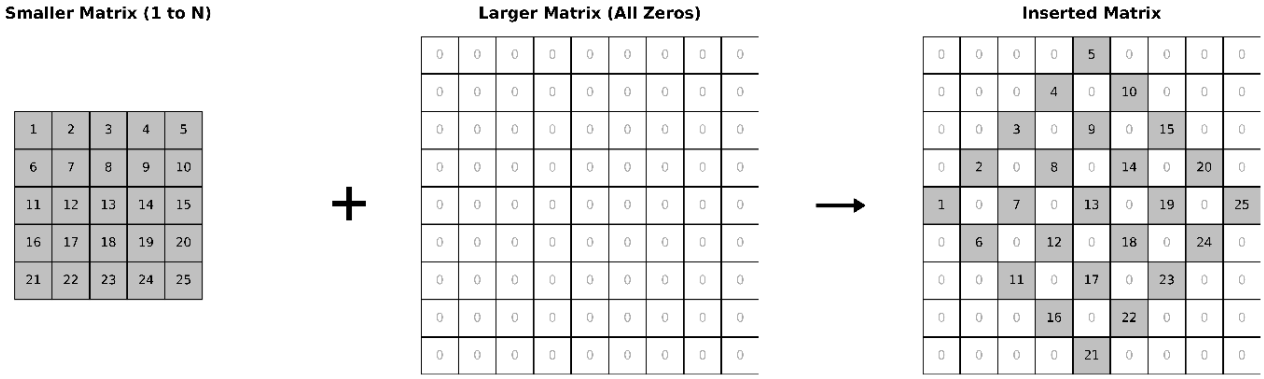
- If the larger matrix is too small to accommodate the pattern, the program must print and return "Impossible".
- Otherwise, return the updated larger matrix.

Pattern Explanation:

If S is a 5×5 matrix, its elements must map to L such that:

- $S_{0,0}$ maps to the left-most tip of the diamond.
- $S_{0,4}$ maps to the top-most tip.
- $S_{4,0}$ maps to the bottom-most tip.
- $S_{4,4}$ maps to the right-most tip.

(Do not use hard-coded coordinates. Derive the mathematical mapping $\big((i, j) \rightarrow (row, col)\big)$.



Section 1: Pixel Resolution

This section focuses on spatial resampling, interpolation mathematics, and basic pixel-by-pixel transformations.

1.1 Pixel Resolution & Interpolation

This section covers techniques for changing the resolution of images – both reduction (downsampling) and increasing (upsampling).

1. **Downsampling:** Reduce the resolution of a high-resolution image by factors of $N = 2, 4$ and 8 by directly sampling pixels.
2. **Upsampling:** Reconstruct the downsampled images back to their original size using your own from-scratch implementations of the following interpolation kernels:
   - **Nearest Neighbor Interpolation:** This is the simplest method. Each pixel in the upsampled image takes the value of its nearest neighbor in the downsampled image.
   - **Bilinear Interpolation:** Compute the weighted average of the 4 nearest known pixels.

     Weight function:

$$K_{\text{bilin}}(t) = \begin{cases} 1-|t|, & |t| \leq 1 \\ 0, & |t| > 1 \end{cases}$$

     For target (x, y) let $i_0 = \lfloor \frac{x}{N} \rfloor$, $j_0 = \lfloor \frac{y}{N} \rfloor$.

     Then

$$I_{\text{up}}(x,y) = \sum_{m=0}^{1} \sum_{n=0}^{1} I_{\text{down}}(i_0+m, j_0+n) \times K_{\text{bilin}}\left(\frac{x}{N} - (i_0+m)\right) \times K_{\text{bilin}}\left(\frac{y}{N} - (j_0+n)\right)$$

     The result is smoother than nearest-neighbor, with linear blending across edges.

- $\boldsymbol{I_{down}(x, y)}$**:** The intensity (color value) of a pixel at coordinates (x, y) in the downsampled image. This represents the original pixel data after the image has been reduced in size.

- $\boldsymbol{I_{up}(x, y)}$**:** The intensity (color value) of a pixel at coordinates (x, y) in the upsampled image. This is the new pixel value you are calculating as part of the reconstruction process.

- **N:** The downsampling factor. If N = 2, the image is reduced to half its original size in each dimension. N=4 reduces it to a quarter, and so on. It determines how much we shrink the original image.

- **t:** A variable representing the fractional distance between a target pixel's location and the location of a neighboring pixel used in the interpolation calculations. It's a normalized value between 0 and 1 (or sometimes extends slightly beyond 1) and is used to weight the influence of each neighboring pixel.

- $\boldsymbol{K_{bilin}(t)}$**:** The bilinear interpolation kernel (weight function). This function determines the weight assigned to a neighboring pixel during bilinear interpolation, based on the distance t.

- **Bicubic Interpolation:** Estimate the pixel value using the 16 nearest neighbors and the cubic convolution interpolation algorithm.

  Concept: Use a 4×4 neighborhood (16 pixels) and a cubic convolution kernel along each dimension.

  The weight function is:

$$K_{\text{cubic}}(t) = \begin{cases} (1.5 \mid t \mid^3 - 2.5 \mid t \mid^2 + 1), & \mid t \mid \leq 1 \\ (-0.5 \mid t \mid^3 + 2.5 \mid t \mid^2 - 4 \mid t \mid + 2), & 1 < \mid t \mid < 2 \\ 0, & \mid t \mid \geq 2 \end{cases}$$

  For a target (x, y) let $i_0 = \lfloor \frac{x}{N} \rfloor$, $j_0 = \lfloor \frac{y}{N} \rfloor$.

  Define the fractional parts:

$$dx = \frac{x}{N} - i_0, dy = \frac{y}{N} - j_0$$

  Now compute:

$$I_{\text{up}}(x, y) = \sum_{m=-1}^{2} \sum_{n=-1}^{2} I_{\text{down}}(i_0 + m, j_0 + n) \times K_{\text{cubic}}(dx - m) \times K_{\text{cubic}}(dy - n)$$

  This double sum uses the 16 nearest neighbors (indices -1 to $+2$ in each direction).

  The kernel decays smoothly to zero at $\mid t \mid = 2$, which guarantees continuity of first-order derivatives and yields the smoothest interpolation of the three methods.

- *Report:* Display the outputs side-by-side with zoomed-in patches to highlight the differences in edge jaggedness, smoothness, and blurring. Calculate and compare the **MSE** and **PSNR** for all three methods.

$K_{\text{cubic}}(t)$: The bicubic interpolation kernel (weight function). A more complex weight function used in bicubic interpolation, also based on the distance t.

dx and **dy:** The fractional distances between a target pixel (x, y) and the nearest integer pixel coordinates (i0, j0). Used specifically in the bicubic interpolation calculations.

Section 2: Point Operations (Intensity Transformations)

2.1 Point Operations

Implement the following point operations. For **each** transformation, you must plot the mathematical transformation curve $s = T(r)$ alongside the original and enhanced images.

1. **Image Negative:** $s = L - 1 - r$ (for both grayscale and colored images).
2. **Logarithmic Transformation:** $s = c\log(1 + r)$. Apply this to an image with a high dynamic range (e.g., a Fourier spectrum image) to reveal hidden details.
3. **Power-Law (Gamma) Correction:** $s = cr^{\gamma}$. Apply this to both an under-exposed (dark) and over-exposed (washed-out) image using appropriate γ values. Explain mathematically why certain γ values expand dark regions while others compress them.
4. **Piecewise-Linear Transformation (Contrast Stretching):** Enhance image contrast by stretching the range of intensity values, design a transformation function with at least 3 segments (e.g., darkening the shadows, stretching the mid-tones, clipping the highlights).

2.2 Bit-Plane Slicing

- Extract and display the 8 individual bit-planes of an 8-bit grayscale image.
- Reconstruct the image using only the top 4 Most Significant Bits (MSBs). Compare the result with the original visually and using MSE.
- Reconstruct the image using only the 4 Least Significant Bits (LSBs). Compare the result with the original visually and using MSE.

Section 3: Step-by-Step Histogram Equalization

This section walks through the progression of histogram-based contrast enhancement, from basic to adaptive.

3.1 Histogram Calculation & Plotting

- Write a function from scratch to calculate the histogram $h(r_k)$ and the normalized Cumulative Distribution Function (CDF) $p(r_k)$ of a grayscale image.
- Plot the histogram as a bar chart. (Do **not** use built-in functions like cv2.calcHist or np.histogram; you must count the pixels yourself, though you can use plt.bar to plot your resulting array).

3.2 Step 1: Global Histogram Equalization (GHE)

- Use the CDF computed in 2.1 to map the original intensity values to new equalized values, forcing a flat histogram.
- *Report:* Apply this to a low-contrast image. Display the original image/histogram alongside the equalized image/histogram.

3.3 Step 2: Local Histogram Equalization (LHE)

- Global HE often washes out small, localized details. Implement **Local HE** using a sliding window (e.g., 15×15 neighborhood).
- For every single pixel, compute the histogram of its neighborhood, find the local equalization mapping, and update the center pixel.
- *Report:* Apply this to an image containing hidden local details. Discuss the computational cost and the "noise enhancement" effect compared to GHE.

Section 4: Histogram Matching & Color Spaces

This section deals with forcing an image to adopt a specific statistical distribution from a reference image, and the challenges of applying these methods to color.

4.1 Image-to-Image Histogram Matching (Specification)

- Implement Histogram Specification from scratch.
- Given a **Source Image (A)** and a **Reference Image (B)**, transform the intensity values of Image A so that its resulting histogram matches the histogram of Image B.
- *Process:* Compute the CDF of Image A ($T(r)$) and the CDF of Image B ($G(z)$). Map the pixels of A to the new values using the inverse mapping $z = G^{-1}(T(r))$.
- *Report:* Show Image A, Image B, the output matched image, and all three histograms. Plot the transformation curve that achieved this mapping.

4.2 Point Operations on Color Images

- Applying HE or Gamma correction directly to the R, G, and B channels independently alters the color balance (hue) of the image.
- Implement an **RGB to HSV** and **HSV to RGB** conversion mathematically from scratch.
- Apply your Global Histogram Equalization (from Section 2.2) **only** to the V (Value/Intensity) channel of an underexposed color image, then convert it back to RGB.
- *Report:* Visually compare this result to applying HE to the R, G, and B channels independently.

Deadline: 10 April 2026

Instructions & Deliverables

- **Allowed Libraries:** You may use numpy, matplotlib, and cv2 / PIL (**ONLY** for reading/writing/displaying images, e.g., imread, imwrite, imshow).
    - **Strict Prohibition:** All core algorithms (Interpolations, HSV conversion, Histograms calculations, Transformations, Matching) **must be written entirely from scratch**. Built-in functions like cv2.resize, cv2.cvtColor, cv2.equalizeHist or cv2.createCLAHE are forbidden and will result in a zero for that section.
- **Report Format:**
  A high-quality academic PDF report is required:
    - **Ch.1 Problem Definition:** Brief explanation of the tasks.
    - **Ch.2 Methodology:** Explanation of your algorithms, mathematical formulas used (e.g., Bicubic weights, histogram mappings), and edge-case handling.
    - **Ch.3 Results & Discussion:** Visual outputs, charts (Histograms, $s = T(r)$ curves), and analytical discussions comparing the methods.
- **Submission:** Submit a single .zip file containing your well-commented Python scripts or Jupyter Notebooks and the PDF report.
  (Naming format: "Student1_Student2_Student3.zip")